TASK4

1.首先需要uav和usv都写一个能记录之前比赛进程的程序，目的是记录机器实时处于什么progress，方便当接受到task4命令的时候，能够清楚自己停在哪一步，之后等task4结束后便可恢复这个进度。

如果大家上报progress的程序已经写好的话，就只需要增加把这份progress的信息储存在本机的代码了，比如是一条一条的：

代码块

```
{
"vehicle": "uav",
 "task": "TASK3",
 "progress": "PICK",
 "error": None,
 "command": None,
 }
```

2.当task4来的时候，主规划器会通过/tongji_mrl/tsky/task_cmd和/tongji_mrl/twave/task_cmd给uav和usv发送第一条信息；但是我们目前挑战的task4难度只需要一个vehicle去执行任务，以下以假如发给usv为例

To usv：

代码块

```
{
  "vehicle": "usv",
  "task": "TASK4",
  "progress": null,
  "error": null,
  "command": {
    "type": "TASK4_BEGIN",
    "command_seq": 100,
    "target_vehicle": "uav",
    "position": {
      "latitude": 31.000001,
      "longitude": 121.000001
    }
  }
}
```

3.要求的是usv接收到这条消息后，去记住position信息；然后，调用“1”中的代码，去读目前处于哪一个任务的哪一个步骤，通过/tongji_mrl/tsky/checkpoint和/tongji_mrl/twave/checkpoint去返回给主规划器，例如：

usv：

代码块

```
{
  "vehicle": "usv",
  "task": "TASK1",
  "progress": "SEARCH_TARGET",
  "error": null,
  "command": "checkpoint"
}
```

4.然后再发一条表示“接收到task4进行”的信号，通过/tongji_mrl/tsky/progress和/tongji_mrl/twave/progress

usv：

代码块

```
{
  "vehicle": "uav",
  "task": "TASK4",
  "progress": "TASK4_ACK",
  "error": null,
  "command": null,
}
```

5.发完上面两条信息后，就可以开始执行任务了，uav和usv的任务都是基于上述读取的position的位置，去setpoint；当到了之后，就再向主规划器发信息，通过/tongji_mrl/tsky/progress和/tongji_mrl/twave/progress

usv：

代码块

```
{
  "vehicle": "usv",
  "task": "TASK4",
  "progress": "TASK4_READY",
  "error": null,
  "command": null,
```

```
}
```

6.然后就等待主规划器给机器发送，继续执行哪个任务的第几步的信息，主规划器会通过/tongji_mrl/twave/task_cmd和/tongji_mrl/tsky/task_cmd

usv：

代码块

```
{
  "vehicle": "usv",
  "task": "TASK1",
  "progress": "SEARCH_TARGET",
  "error": null,
  "command": "resume"
}
```

NOTE：task4只会给一个机器发，但是例如只有usv需要执行的时候，uav也要原地等候，暂停任务，并且去上传checkpoint;但是不需要上传"TASK4_ACK"和"TASK4_READY"的信息；